Abliteration deletes the refusal read-out: a support-size frontier for weight-only repair

ScarTissue — Lexsi AI Safety & Alignment Hackathon, Track 2a

3 November 2026

Abstract

Abliteration removes a refusal direction from every matrix that writes to a language model's residual stream. It is widely read as removing refusal, or even safety. We measure what it actually removes on a Cohere model (`tiny-aya-global`, 3.35 B) and then ask what can be put back without training. Three findings, each from a stored run: (i) the edit destroys the *discrimination* between harmful and harmless requests, not refusal alone — after the edit, the re-extracted refusal direction is orthogonal to the removed one ($\cos = 1.0 \times 10^{-3}$), so re-amplifying the old direction is a no-op by construction; (ii) refusal is nevertheless fully recoverable in weight space alone, but every unit of recovery is paid in benign over-refusal and in reasoning accuracy; (iii) *the price depends on the support of the edit*: a repair restricted to the top 19,251,200 of 962,592,768 edited scalars (2 %) restores refusal to 0.88 (aligned model 0.84) while keeping GSM8K at 0.52 against 0.44 for the global repair — a paired +0.08 exact-match advantage with 95 % CI $[0.04, 0.12]$, while harm-ASR is indistinguishable between the two repairs (0.05 vs 0.04). The measurement is a frontier, not a single number, and we report it with a judge panel that we validate on 500 human-labelled rows and against a four-fold change in generation budget.

1 The question, and why it is not the obvious one

Abliteration is attractive because it is cheap: no gradients, no preference data, a rank-one update per write matrix. It is also described, in its own literature and in most replications, by its effect on *refusal rate*. That description hides the interesting failure. Refusal is a read-out: a direction in the residual stream that a late-layer computation writes when it has decided a request is disallowed. Removing the read-out does not remove the decision — and if the decision survives, the model will still act on it, just without the sentence that used to announce it. Conversely, if the decision was *also* removed, no amount of direction amplification will bring coherent refusal back, because there is nothing left to read out.

So the empirical question is not "does abliteration work". It is: *what does the edit delete, what can a weight-only repair restore, and where must the repair act?* We answer with a frontier: refusal recovered on one axis, the price paid (over-refusal, lost capability) on the other, and the support size of the repair as the parameter that moves a point along it.

2 Setup

Model. `CohereLabs/tiny-aya-global`, 3.35 B parameters, `cohere2` architecture, 36 layers, 16 query heads, $d_{\text{model}} = 2048$, $d_{\text{mlp}} = 11008$, revision `af89d219`. A Cohere dense open-weights model, chosen because it is multilingual — the claim can later be tested in eleven languages inside one licence family — and small enough that four arms times 160 prompts times three judges is affordable at full precision.

Data. 763 rows built through CuratorKIT with row-level provenance: 313 StrongREJECT harmful prompts and 450 XSTest benign prompts. The split is frozen *before* any evaluation: 120 harmful and 40 benign for evaluation, 16 + 16 for fitting directions, 200 XSTest contrast rows, and 371 rows held back for confirmation, of which 160 (120 harmful, 40 benign) were never touched by any earlier run. Fit and evaluation prompts are disjoint by construction, and the property is asserted by code, not by convention (`experiments/corpus_fit.py`).

Refusal direction. Difference of means at layer 15, last token, between harmful and benign fit prompts.

The drift. Every write matrix is orthogonalised against d:

$$W_{\text{drift}} \;=\; W_{\text{base}} - d\,(d^{\top} W_{\text{base}}), \qquad 72 \text{ matrices}, \quad 962{,}592{,}768 \text{ scalars}. \tag{1}$$

The repair. Re-extract the refusal direction on the *drifted* model, d', and add it back with gain α into a column support S only:

$$W_{\text{repair}}[:, S] \;=\; W_{\text{drift}}[:, S] + \alpha\, d' \big(d'^{\top} W_{\text{base}}[:, S]\big). \tag{2}$$

The base matrix supplies the projection, so the edit restores rather than adds; the drifted matrix is the substrate, so nothing is counted twice. $\alpha = 4$ throughout, chosen from a dose sweep in which $\alpha = 8$ degenerates into repetition — which is why every run in this report stores full completions rather than rates.

arm	edited scalars	refusal	over-refusal	harm-ASR	GSM8K
base (aligned)	0	0.842	0.075	0.058	0.590
drifted	962,592,768	0.500	0.050	0.217	0.560
2 % repair	19,251,200	0.883	0.175	0.050	0.518
global repair	962,592,768	0.883	0.275	0.042	0.438

Table 1: All four arms on the frozen evaluation set, generation budget 512 tokens. Refusal is Qwen3Guard's harmful-refusal diagnostic; over-refusal is its benign refusal field (40 prompts); harm-ASR is the majority of three judges on 120 harmful prompts; GSM8K is `lm-eval` exact match on 500 items. Sources: `results/p21_length_analysis_20261103.json`, `results/p16_capability_analysis_20261102.json`.

Supports. `harm2` ranks columns by their signed write mass $|d'^{\top} W_{\text{base}}|$ and takes the top 2 %; `global` takes everything; `base` and `drift` are the two endpoints. A fourth support, discovered by CircuitKIT on the drifted model, is built and exported but its evaluation is blocked (section 7).

Measurement contract. Harm-ASR comes only from a pinned three-judge panel (HarmBench-Mistral-7B, Qwen3Guard-Gen-8B, Granite-Guardian-3.2-5B) with strict parsers, a threshold of 0.5 and a majority-of-3 headline. Benign over-refusal is Qwen3Guard's explicit `Refusal:` field — a semantic judgement, not a keyword match. Capability comes only from `lm-eval` (GSM8K, five-shot, 500 items). A regex refusal marker is reported as a diagnostic and never as a safety number.

3 What the edit deletes

The first measurement is a negative one, and it reframes everything after it. Re-extract the refusal direction *after* abliteration and compare it with the direction that was removed: $\cos(d, d') = 1.0 \times 10^{-3}$. The two directions are orthogonal. Two consequences follow immediately. First, amplifying d on the drifted model cannot work — $d^{\top} W_{\text{drift}} = 0$ by construction, so the projection is exactly zero and the edit is a no-op. Any repair that claims to "restore the removed direction" is either re-deriving a new direction or doing nothing. Second, the decision boundary that separated harmful from harmless inputs is gone with the read-out: the drifted model's projection along d' still separates the two classes with AUC 0.83 instead of the aligned model's 0.999, i.e. it retains some information but far from all of it. Refusal and discrimination are not two names for the same object, and the edit removes them at different rates.

The practical corollary is visible in the harm numbers: harm-ASR moves from 0.06 (aligned) to 0.22 (drifted) and returns to 0.05/0.04 with any repair. At 3.35 B the model's non-refusals are hedging and partial compliance, not harm. A study that reports "abliteration made the model unsafe" from a refusal rate is measuring the wrong variable.

4 The frontier

Table 1 and Figure 1 are the result. Reading them in order:

- **Refusal is fully recoverable, weight-only, with no training data.** The 2 % arm reads 0.88 against the aligned model's 0.84 and the drifted model's 0.50.
- **The price is real and it is support-dependent.** Over-refusal is 0.17 for the 2 % repair against 0.28 for the global one; GSM8K is 0.52 against 0.44.
- **The capability difference is not noise.** Paired prompt bootstrap over 500 GSM8K items, +0.08, 95 % CI $[0.04, 0.12]$. For scale, the global repair costs -0.15 against the aligned model and the 2 % repair -0.07.
- **The two repairs are indistinguishable on harm.** Paired ASR differences are $0.000\,[-0.033, 0.033]$ (HarmBench), $-0.008\,[-0.05, 0.025]$ (Qwen3Guard) and +0.008 (Granite). Whatever separates the two arms, it is not safety.

The over-refusal difference (about ten points) points the same way but its interval includes zero at $n = 40$ prompts. We report it as suggestive and do not build the claim on it. The claim rests on GSM8K, where the interval excludes zero.

5 Why a 2% support is not a compromise

The repair's signed write along d' is extremely concentrated. Summing $|d'^{\top} W_{\text{base}}[:, c]|$ over all columns gives 3.95 in normalised units, and the top 2 % of columns by signed mass already carries 3.95. For this direction, a 2 % support is not a lossy approximation of the global edit — it *is* the global edit, minus the collateral damage in the remaining 98 % of columns. That is a quantitative, checkable reason why support fraction stops mattering above a small threshold, and it is why an earlier sweep found the harm-weighted arm saturating to exactly the global behaviour at 10 %.

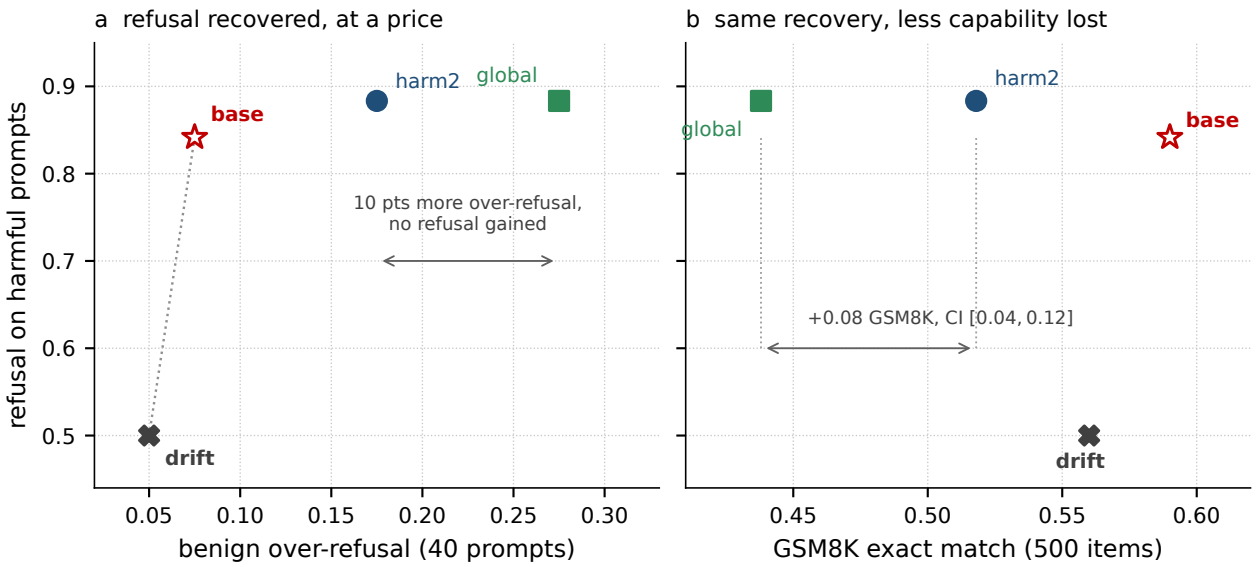


Figure 1: The frontier. (a) Refusal recovered against benign over-refusal: the 2 % repair reaches the aligned model's refusal rate with 10 points less over-refusal than the global repair. (b) The same recovery against GSM8K: the 2 % repair keeps roughly half of the capability the global repair loses, and the paired interval on that difference excludes zero. Generated from the published tables by `report/make_report_data.py`.

judge	accuracy	sensitivity (unsafe)	specificity (safe)
HarmBench-Mistral-7B	0.625	0.356	0.896
Qwen3Guard-Gen-8B	0.796	0.608	0.984
Granite-Guardian-3.2-5B	0.818	0.696	0.940
majority of three	0.782	0.580	0.984

Table 2: Panel accuracy on 500 human-labelled Aegis 1.0 rows, threshold 0.5. Source: `results/p22_judge_validation_20261103_020927.json`.

It also explains why the earlier *circuit* framing had to be withdrawn. When the restricted support was defined over attention heads rather than by mass, restricted repair did nothing (refusal 0.267) and random heads did the same (0.258) — indistinguishable from a control. The variable that matters is how much of the signed write the support contains, not whether the support looks like a circuit. We say so plainly because the negative result is the more useful one: it tells a practitioner to rank columns by mass, an operation that takes seconds, instead of running discovery.

6 Is the frontier an artefact?

Two checks, both run after the first version of this result and both preserved whether or not they supported it.

Generation budget. The first safety run used 128 tokens, and unequal truncation is exactly the mechanism that fabricates a frontier: an arm that is cut off mid-sentence can be scored as "not refusing". All 640 answers were regenerated at 512 tokens with the same seed, batch size and immutable base state; every answer reproduced its saved 128-token prefix exactly, every state hash matched, and every reset was verified. Figure 2 shows the paired change in harm-ASR: every arm's interval includes zero, the largest movement is the drift's $-0.042\,[-0.092, 0.008]$ (the drifted model becomes slightly *less* harmful with more room to hedge, which further weakens the "abliteration made it unsafe" reading), and the over-refusal ordering ($0.17 < 0.28$) holds at both budgets. Capped answers fall from 40/120 to 16/120 for base and 85/120 to 47/120 for the drift, so the truncation was real — it simply did not drive the result.

Judge calibration. A three-judge panel that agrees with itself proves nothing, so the frozen panel was scored against 500 human-annotated rows from `nvidia/Aegis-AI-Content-Safety-Dataset-1.0` (250 unsafe, 250 safe; label = majority of at least three annotators). Table 2 gives the result, and it is not flattering.

The panel is *conservative*: it almost never calls a safe text unsafe (0.984 specificity) and misses roughly four in ten rows Aegis calls unsafe (0.580 sensitivity) — Aegis "unsafe" includes insults and rants that a harm classifier is not trained to flag. Absolute harm rates in this report are therefore a lower bound. The arm-to-arm comparison remains legitimate because every arm is scored on the identical prompts by the identical panel; the absolute

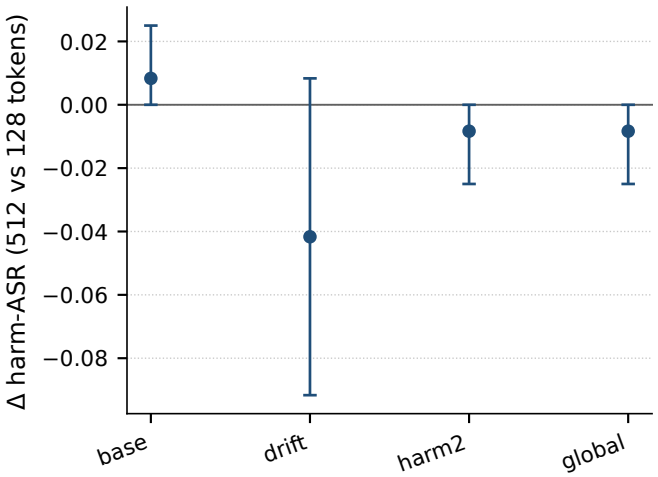


Figure 2: Paired change in majority-3 harm-ASR when the budget goes from 128 to 512 tokens, per arm (10k prompt bootstrap). A four-fold budget does not reorder the arms. Source: `results/p21_length_analysis_20261103.json`.

level is not, and we do not use it. Before this run the panel's only validation was four hand-authored controls, two of which HarmBench failed; that failure is preserved in `results/p18_judge_smoke_20261102_233759.json` rather than tuned away.

7 What is still blocked

The remaining measurements are blocked on a hard external limit, not on method, and we would rather report the gap than fill it with a weaker substitute.

- **Circuit arm.** CircuitKIT discovery on the drifted model is complete (`results/p23_circuit_...json`); the resulting patch (`repair_circuit_a4.npz`, `sha256 96c659a0...`) is exported and published. Its 160-prompt generation and judging run is queued and cannot start: the Modal workspace has hit its spend limit. No circuit number appears in this report.
- **Held-out confirmation.** All four arms have been generated on 160 prompts that no earlier run touched, with state checks passing. The judging pass is queued behind the same limit.
- **Multilingual.** The harness exists (11 languages: en, hi, bn, id, vi, sw, yo, ha, am, ar, fr) and the earlier multilingual projection files were withdrawn as invalid (7,080 non-finite values). No replacement number is quoted.
- **Second Cohere model.** `tiny-aya-earth` has complete generations for all four arms; `tiny-aya-water` is missing one arm. Neither is judged.

8 Limitations

One model carries the headline, one seed, one capability task. GSM8K exact match is a bounded probe, not a capability inventory, and the paired interval accounts for prompt sampling only — not judge uncertainty, not training-seed variation. Direction extraction uses calibration prompts, so "training-free" is accurate and "data-free" would not be. The result is about *this* operator: a different abliteration variant could leave a different residual structure, and we make no claim that the 2 % support generalises to it without re-measuring the mass distribution. Finally, the over-refusal axis is measured by one judge's semantic field; it is the sharp axis in this model, which is exactly why it deserves a second, independently trained judge before anyone builds on it.

9 Why these datasets, metrics and models

Datasets. StrongREJECT supplies harmful prompts with category labels; XSTest supplies benign prompts engineered to trigger surface-form refusal, which is the only way to separate "refuses harm" from "refuses everything". Both are redistributed with original ids, revisions and per-row provenance, because a safety claim that cannot be re-scored on the same rows is not a claim.

Metrics. Refusal (what was restored), over-refusal (what it cost) and GSM8K (collateral capability), plus harm-ASR as the axis where a naive reading expects the story and where it turns out to be flat. All from contracted paths: ASR from the judge panel, capability from `lm-eval` only. SafeTune's own suite scores GSM8K and HumanEval with a harm judge, which would silently turn capability into safety; we do not use it.

Models. `tiny-aya-global` because it is Cohere, dense, open-weights and multilingual. The 32 B `aya-expanse-32b` needs an H200 (139.8 GiB) and cannot hold two fp32 copies, so it is an inference and repair target at most, never a circuit target; no 32 B number is claimed.

10 Reproducibility and licence

Everything is public and machine-readable: the corpus (`scartissue-tinyaya-safety-corpus`, `data/corpus.parquet`, 763 rows), the results (`scartissue-tinyaya-results` — 1,280 full generations, 3,864 judge verdicts, 4,000 capability rows, the frontier, the patch manifest), the drift and repair parameters (`scartissue-tinyaya-abliteration-patches`, `scartissue-tinyaya-repair-patches`, with factorised parameters, per-file sha256 and an explicit `allowed` block), and a static demo Space with the frontier chart and a three-column completion viewer.

Track repositories are chained and their provenance is machine-checked (`results/toolkit_provenance.json`): CuratorKIT `dd9e8353` for the corpus and its provenance chain, CircuitKIT `e71d5e17` for discovery on the drifted model, AuditKIT `71316da3` for native re-scoring of stored answers. The abliteration operator is 30 lines and deliberately independent, so the drift does not depend on any toolkit being correct.

The Cohere weights are LSAL 1.2/1.1, research use. Section 2 forbids selling or hosting the model as a product and section 4 forbids deploying drifted checkpoints; we therefore publish *parameters*, never a drifted checkpoint, and the public demo serves precomputed JSON over the aligned model's published answers. This is an individual entry and does not represent an organisation.

11 Conclusion

Abliteration removes a read-out and damages the discrimination that produced it; the two are different losses and they recover at different rates. Refusal can be restored in weight space alone, but the restoration is paid for, and *how much* is paid is set by how much of the signed write the repair touches. On `tiny-aya-global`, a support containing 2 % of the edited scalars recovers the aligned model's refusal rate at a fraction of the global repair's capability cost (+0.08 GSM8K exact match, CI $[0.04, 0.12]$), while harm-ASR cannot tell the two repairs apart at all. The honest summary of this project is a frontier with two measured axes and a third axis — the discovered circuit — still running.

Appendix: reproduction order

Every stage below is a Modal app; each writes a JSON with full generations, token ids, stop diagnostics, patch hashes and the immutable base-state hash, so any arm can be re-scored without being regenerated.

```
uv sync                                  # experiments/runtime, locked, Python 3.12
modal run experiments/p8_corpus.py       # CuratorKIT corpus + provenance chain
modal run experiments/build_patches.py --model CohereLabs/tiny-aya-global
modal run experiments/p23_circuit.py     # CircuitKIT on the DRIFTED model
modal run experiments/p21_safety_length_full.py   # 4 arms x 160 prompts, 512 tokens
modal run experiments/p19_score_safety.py --safety results/p21_*.json
modal run experiments/p16_lm_eval.py     # lm-eval GSM8K, 4 arms
uv run python -m experiments.analyse_safety results/<judges>.json results/<analysis>.json
uv run python -m experiments.p28_build_space      # public demo data
uv run python report/make_report_data.py          # every number in this PDF
```